

Dev.ITISYOU — Deep Product Architecture and Verified Developer Knowledge Research

Research report for Leela (ITISYOU network). Prepared 21 August 2026. This is a research document, not an implementation plan.

Recommended filename: `01-CLAUDE-DEV-KNOWLEDGE-ARCHITECTURE-RESEARCH.md`

Every claim is labelled: **FACT** (verifiable, sourced), **RESEARCH FINDING** (from a study/source, may be contested), **INTERPRETATION** (my reasoning), **RECOMMENDATION** (what to do), **UNKNOWN** (unresolved).

Executive Summary

INTERPRETATION. The timing thesis is strong; the execution risk is severe. The category Dev.ITISYOU targets — structured, testable, version-aware troubleshooting procedures with reproduction evidence — is genuinely under-occupied. No named product in 2025–2026 combines diagnostic decision trees + reproduction evidence + version-staleness tracking. But the same forces that make the gap exist (AI answers being fast, free, and "good enough") also make the contribution cold-start problem brutal. The hardest, most defensible pillar (a reproduction-evidence graph) is also the hardest to bootstrap.

FACT. Stack Overflow question volume has collapsed. Per data first reported in 2025 (Stack Overflow Data Explorer / top-1% contributor Theodore R. Smith), posts fell 64% and daily active users 47% between April 2024 and April 2025; by February 2025 SO received only 29,693 new questions — the lowest monthly total since 2010 — and monthly volume has returned to roughly its 2008 launch level. Developers use AI heavily (84% use or plan to use AI tools) but trust it little (46% actively distrust output accuracy; only 3% "highly trust" it).

INTERPRETATION. The wedge is not "answers" (AI owns that) but "evidence that a fix actually works, on a specific version, verified by someone other than the author." That is precisely what AI cannot fabricate honestly and what Stack Overflow never structured. The MVP must test one thing: will developers trust and contribute reproduction evidence?

Core Product Hypothesis

The hypothesis: Stack Overflow stores answers; AI generates possible answers; Dev.ITISYOU should store structured, testable, version-aware engineering procedures with evidence about whether they actually work.

INTERPRETATION. This is a real and defensible distinction *if and only if* the evidence layer is trustworthy and cheap to produce. The concept maps cleanly onto three mature bodies of practice: (1) SRE runbooks and runbook automation; (2) scientific reproducibility and artifact evaluation; (3) evidence grading from medicine. The novelty is combining them for developer troubleshooting. The risk is that each of those three fields is individually hard, and this product needs all three at once.

Evidence For

FACT. Trust in AI is low and falling: Stack Overflow's 2025 survey (49,000+ respondents, 177 countries) found 46% distrust AI accuracy vs. 33% who trust it; experienced developers are most cautious (only

2.6% "highly trust", 20% "highly distrust"). 66% of developers struggle with AI solutions that are "almost right but not quite", and 45% say debugging AI-generated code takes longer than writing it themselves.

FACT. In a future with advanced AI, the #1 reason developers say they would still ask a human is "When I don't trust AI's answers" (75%). This positions verified human evidence as durable.

FACT. Stack Overflow itself is pivoting to "Knowledge-as-a-Service" and launched Stack Overflow for Agents (beta, 2026) to close what it calls the "Ephemeral Intelligence Gap" — agents repeatedly rediscovering the same fixes in isolation. This validates the "structured, reusable, verified knowledge" thesis at the incumbent level.

RESEARCH FINDING. The competitive niche is genuinely under-occupied. Runme/Stateful makes runbooks executable; Doc Detective tests that docs match reality; Context7 and Tessel provide version-specific docs/specs to AI; Dosu builds "living" codebase knowledge. None combine diagnostic trees + reproduction evidence + version staleness.

INTERPRETATION. There is a genuine white space around a "reproduction-evidence graph" for developer fixes.

Evidence Against

FACT. Phind — a developer answer engine with cited sources — raised a \$10.4M Series A dated 3 December 2025 (led by Bessemer Venture Partners), then shut down abruptly on 16 January 2026 with no sunset period, deleting user data by 30 January 2026. Its niche was "eroded by improvements in GitHub Copilot, GPT-4, and other general-purpose LLMs"; search interest peaked at over 27,000 monthly searches in early 2024 then shed nearly 91% over the following two years as Copilot, Claude, and ChatGPT absorbed the same coding-question use cases. **INTERPRETATION.** "Better cited developer answers" alone is not defensible against general LLMs. Dev.ITISYOU must not be a cited-answer engine.

RESEARCH FINDING. Reproducibility is hard even with strong incentives. Pimentel et al. (2019), studying 1.4M GitHub Jupyter notebooks, found that of 863,878 attempted executions of valid notebooks, only 24.11% executed without errors and only 4.03% produced the same results. A biomedical study (Samuel & König-Ries, arXiv:2308.07333) successfully reproduced only three of 22 notebooks from five PubMed Central publications. **INTERPRETATION.** If well-motivated scientists cannot make code reproduce, expecting volunteer developers to produce reproducible evidence at scale is optimistic.

RESEARCH FINDING. ACM artifact evaluation is optional, occurs after acceptance, and checks whether code *can* produce results, not whether it *did* — a known structural weakness. **INTERPRETATION.** Even formal reproducibility badging is gameable; a community system will be more so.

INTERPRETATION. Contribution cold-start is the existential risk. Reproduction evidence is worthless until many people reproduce, but nobody reproduces until the content is valuable. This is a two-sided chicken-and-egg problem harder than Stack Overflow's original one, because verification is more work than answering.

INTERPRETATION. Incumbents with distribution (Stack Overflow, Sourcegraph, Dosu at 50,000+ projects, Warp) can bolt on evidence features faster than a solo builder can bootstrap a community.

Knowledge Architecture Findings

RESEARCH FINDING. Each candidate model has known strengths and failure modes:

Q&A (Stack Overflow): great for discovery and one-off answers; poor at versioning, staleness, and multi-step diagnosis. Answers rot silently.

Wiki: good for canonical prose; weak provenance, weak structure, edit wars.

Graph: models relationships (Problem↔Tech↔Version↔Symptom↔Test↔Fix↔Evidence) naturally; higher authoring/query complexity.

Decision tree / troubleshooting tree: matches the diagnostic flow directly; brittle with loops, uncertainty, and multiple plausible causes.

Runbook: proven in SRE/ops (PagerDuty Runbook Automation, Rundeck); linear, execution-oriented, weaker at branching diagnosis.

Diagnostic protocol: strongest match to "test → PASS/FAIL → next test"; expensive to author well.

Knowledge card: good atomic unit for retrieval and reuse.

Executable documentation (Jupyter/literate programming, Runme): supports verification but reproducibility is not automatic.

Versioned documentation: essential given staleness, but pure docs lack evidence.

RECOMMENDATION. Model the domain as a **graph of typed, versioned nodes** (the conceptual entities in the brief: Problem, Symptom, Environment, Diagnostic Test, Root Cause, Fix, Playbook, Evidence, Reproduction, etc.), with **Playbook** as the primary composed unit (an ordered/branching arrangement of Tests, Branches, and Fixes) and **Evidence/Reproduction** as append-only leaf records attached to specific Fix+Environment+Version tuples. Do **not** select a database now.

RECOMMENDATION — mutability model:

Immutable / append-only: Reproduction results, Evidence records, test-run outputs, timestamps, author attribution. These are historical facts; they must never be edited, only superseded.

Versioned: Playbooks, Diagnostic Tests, Fixes, Root Cause explanations. Every edit creates a new revision; old revisions remain addressable so past evidence stays bound to the exact text it validated.

Editable (with history): Tags, titles, summaries, cross-links, classification.

Deprecated (state, not deletion): When a fix is invalidated by a new version or security advisory, it is marked deprecated/outdated and retained for provenance, never silently removed.

INTERPRETATION. The single most important architectural rule: **evidence must be bound to an immutable snapshot of the exact procedure it verified.** If a playbook is edited, its old reproductions do not automatically carry forward — they attach to the prior revision. This prevents the Stack Overflow failure mode where an edited answer's upvotes no longer reflect the current text.

Verification Findings

RESEARCH FINDING. Mature evidence-grading and provenance systems provide directly transferable design patterns:

GRADE (medicine; adopted by 110+ organisations including WHO, Cochrane, NICE): rates certainty as High/Moderate/Low/Very Low; evidence *starts* at a level based on study type and is *downgraded* for risk of bias, inconsistency, indirectness, imprecision, and publication bias, or *upgraded* for large effects. Crucially, it separates *certainty of evidence* from *strength of recommendation*.

ACM artifact badges: "Artifacts Available", "Artifacts Evaluated-Functional", "Artifacts Evaluated-Reusable", "Results Reproduced" (independent team, author's artifacts), "Results Replicated" (independent team, own artifacts). Note the key distinction: **Reproduced ≠ Replicated.** [arxiv](https://arxiv.org/abs/2304.01347)⁷

SLSA provenance (build track, levels 0-3, v1.0 April 2023): higher levels require hosted, tamper-resistant builds and signed, unforgeable provenance via in-toto attestations. **INTERPRETATION.** The lesson: a claim is only as trustworthy as the unforgeability of the record behind it.

RECOMMENDATION — what "Verified" should mean. Never use a single "Verified" badge. Use a **multi-layer, GRADE-style confidence ladder**, where each layer is a distinct, independently-earned attestation: AI-structured / AI-reviewed (lowest; never counts as verification)

Automated syntax/lint validation

Automated execution in sandbox (author's environment)

Author tested (self-reported, signed)

Independent reproduction (one other party, different account, ideally different environment)

Multi-environment reproduction (N independent parties across ≥ 2 environments)

Official-documentation-supported (linked primary source)

Maintainer-confirmed (verified project maintainer)

Continuously tested (CI re-runs on a schedule / on dependency change)

RECOMMENDATION — thresholds and confidence:

Minimum bar to show any positive signal: **author-tested + at least one independent reproduction on a stated version**. Author-only claims should be visibly marked "unverified."

Represent confidence as a **discrete band (e.g. Very Low $\rightarrow$ High)**, never a bare percentage, to avoid the small-sample trap. Show raw counts always ("3 of 4 reproductions passed on Node 20"), and suppress or heavily caveat percentages below a minimum denominator (e.g. $n < 5$).

Failures must lower confidence and be first-class. A failed reproduction is evidence, not noise. Store it, show it, and let it downgrade the band.

Version compatibility changes confidence: evidence on v18 says nothing certain about v22. Confidence is scoped per version/environment tuple, not global.

RESEARCH FINDING — anti-gaming. Stack Overflow has detected sockpuppets and voting rings since 2009 using statistical analysis of voting patterns; academic work identifies reputation-fraud indicators and voting-ring structures on SO, and sockpuppet detection on Wikipedia uses authorship/writing-style, network-structure, and propagation-tree features. **RECOMMENDATION.** Reputation must be based on **successful independent reproductions**, not votes; weight reproductions by reproducer independence (different account age, IP/ASN, environment fingerprint) and detect coordinated clusters. Handle **conflicting reproductions** by showing the distribution (not averaging it away) and surfacing environment differences as the likely explanatory variable.

Contribution Findings

INTERPRETATION. Friction is the enemy; verification is inherently high-friction. The resolution is a **tiered contribution model** where the *floor* is trivially easy and *depth* is optional and incentivised.

RECOMMENDATION by contributor type:

First-timers / anonymous: one-click **"this fixed it"** quick contribution (a lightweight reproduction vote bound to a version/environment), plus AI-assisted structuring of a free-form paste. Lowest friction; feeds the evidence graph without demanding authorship.

Experienced engineers: guided form + AI-assisted interview to convert a war story into a structured playbook; import from Markdown; GitHub issue conversion.

Maintainers / official/company contributors: verified identity, "maintainer-confirmed" attestations, and the ability to mark official-doc support. Use a **DCO-style sign-off, not a CLA**, for the contribution license (lower friction; CLAs are known to deter first contributions), under a share-alike content license in the CC BY-SA family so knowledge stays open and attributed. Tenthirtyam

All: reputation accrues from reproductions others could confirm, not from posting volume.

RESEARCH FINDING. Runbook automation tools (PagerDuty/Rundeck) show the enterprise appetite for codifying procedures, but they are execution engines inside trusted infrastructure, not open knowledge bases — confirming the gap for an open, evidence-bearing troubleshooting repository.

Troubleshooting UX Findings

RESEARCH FINDING. Decision-tree/troubleshooting-tree best practices from support-KB literature: keep trees as short as possible; one action per step; branch on user input with clear PASS/FAIL paths; embed verification ("what success looks like") at each step; show a meaningful last-reviewed date and an accountable owner; break overly complex trees into smaller linked trees. Document360 + 2

↗

RECOMMENDATION — handling the hard cases:

Branches: explicit PASS/FAIL edges from each Diagnostic Test.

Loops: allow but cap; detect cycles and offer an escape to "escalate / multiple causes."

Uncertainty & multiple plausible causes: rank candidate root causes by prior probability (from reproduction data) rather than forcing a single path; let users pursue the most-likely branch first.

Missing information: insert optional "gather this signal" steps; never dead-end silently.

Environment differences: make environment a first-class filter so a branch can be scoped ("only on Docker Desktop for Mac").

Dead ends: every terminal node offers "none of these worked → contribute a new branch/evidence," turning failure into contribution.

Versioning / Staleness Findings

INTERPRETATION. Staleness is the core enemy and the core moat. If Dev.ITISYOU tracks *which versions a fix was verified on* and *when*, it can do something neither Stack Overflow nor a raw LLM does: proactively flag knowledge for re-verification.

RECOMMENDATION — staleness triggers. Maintain a machine-readable map from each playbook to the technologies/versions/APIs it depends on, then trigger re-verification when:

a dependency releases a new major/minor version (React, Node, etc.),

a runtime changes behaviour (Cloudflare Workers, Docker networking),

a deprecated API is announced, or

a **security advisory** invalidates a procedure.

RESEARCH FINDING. Context7 already demonstrates demand for version-specific documentation delivered to AI tools, and Tessl's Spec Registry explicitly targets "version blur" and "version mixups."

INTERPRETATION. Both prove the version-awareness need but neither carries reproduction evidence — Dev.ITISYOU's differentiator. Tessl

↗

RECOMMENDATION. Do **not** let AI alone decide correctness after a version change. AI proposes "this may be stale"; a human or automated re-run confirms. AI flags; evidence decides.

Automated Testing Findings

RESEARCH FINDING — sandboxing landscape (2026). For untrusted code execution the consensus is that shared-kernel containers (plain Docker/runc) are insufficient; isolation runs Docker (process) → gVisor (userspace kernel) → Firecracker/Kata (microVM, hardware-level). E2B uses Firecracker microVMs

purpose-built for untrusted/agent code; Modal uses gVisor; Cloudflare, Vercel, Daytona, Northflank and others offer managed sandboxes. Note that 2026 saw the first Firecracker hypervisor-escape-class CVEs, so even microVMs are not infallible.

RECOMMENDATION — tiering by what can realistically be auto-tested:

SAFE FOR MVP: self-contained code snippets, package examples, unit tests, deterministic API-shape checks — run in ephemeral microVM/gVisor sandboxes with no network, strict CPU/memory/time limits, and no secrets. Use a managed sandbox provider rather than self-hosting isolation initially.

POSSIBLE LATER: Docker Compose stacks, database examples, browser behaviour, controlled/edge behaviour — higher cost and orchestration.

TOO DANGEROUS / EXPENSIVE (for now): anything needing cloud credentials, real external services, destructive operations, GPU, or long-lived state; arbitrary network egress.

INTERPRETATION. Automated execution is a *supporting* signal, never the top of the confidence ladder. Independent human reproduction across environments remains the gold standard because most valuable troubleshooting knowledge is environment-dependent and not fully sandbox-reproducible.

Search Findings

RESEARCH FINDING. Hybrid retrieval (BM25 lexical + dense embeddings, fused via reciprocal rank fusion, optionally cross-encoder reranked) consistently outperforms either method alone, often by double-digit margins on recall/nDCG. BM25 excels at exact tokens (error codes like ECONNREFUSED, API names, SKUs) that embeddings dilute; embeddings capture paraphrase. Reranking on hybrid candidate lists beats reranking on either alone.

RESEARCH FINDING. Sentry groups errors by a "fingerprint": it prioritises stack trace, then exception, then message, with custom and built-in fingerprinting rules to merge or split. **RECOMMENDATION.** Implement **error-signature matching** (normalise stack traces/error strings into fingerprints) as a first-class retrieval path alongside hybrid text search and graph traversal. Developers describe problems via error messages, stack traces, natural language, environment metadata, code, and logs; route each to the retrieval mode it suits — fingerprints for errors, BM25 for exact tokens, embeddings for prose, graph traversal for "related fixes on this version."

Trust / Security Findings

RESEARCH FINDING — the threat surface is real and current. 2025–2026 saw major supply-chain attacks. On 8 September 2025, attackers phished npm maintainer Josh Junon ("Qix") via a lookalike domain (npmjs.help, registered three days earlier) and within roughly 16 minutes injected crypto-draining malware into 18 packages — including chalk, debug, ansi-styles, and strip-ansi — with ~2.6 billion combined weekly downloads (direct losses were limited to ~\$500 in crypto). The self-replicating Shai-Hulud worm infected 500+ packages; typosquatting and malicious postinstall scripts remain endemic. The "Trojan Source" attack (Cambridge, 2021) uses Unicode bidirectional/homoglyph characters to make code read one way to humans and compile another way — directly relevant to copy-pasted snippets. Prompt injection is OWASP's #1 LLM risk; stored/indirect injection can hide instructions in user-submitted content.

RECOMMENDATION — research-backed trust controls:

Never auto-execute untrusted fixes outside a locked-down sandbox (no network, no secrets, resource caps).

Scan all submissions for hidden Unicode (bidi controls, homoglyphs, zero-width characters); render them visibly and warn.

Treat every submission as untrusted input to any AI feature: separate untrusted content from system instructions, sanitise, and never let submission text act as instructions (defends against stored prompt injection).

Static-scan snippets for destructive shell commands, credential exfiltration patterns, and known-malicious package references before display.

Flag unsafe package recommendations against known-malicious lists.

Detect fake reproduction accounts / reputation gaming with the sockpuppet/voting-ring techniques above.

Rate-limit and provenance-check to blunt SEO spam, coordinated manipulation, and AI-generated low-quality content.

INTERPRETATION. The verification model is itself the primary anti-abuse mechanism: if reputation flows only from *independently confirmable* reproductions, most spam and gaming have no payoff.

AI Role Findings

RECOMMENDATION. AI is a structuring, triage, and search assistant — never an authority and never a source of verification evidence. Matrix:

Moderation: community flagging + reputation-gated review; automated abuse detection first-pass.

Disputes / conflicting evidence: show the distribution and environment context; do not delete the minority report.

Forks: allow forking a playbook when approaches genuinely diverge; link forks.

Merging duplicates: AI proposes, humans confirm; preserve both histories and re-map evidence carefully.

Ownership / attribution: persistent author attribution per revision; official/company submissions carry verified badges.

Licensing: DCO sign-off + CC BY-SA-family content license (keeps knowledge open, attributed, share-alike; avoids CLA friction).

Deletion: prefer deprecation over deletion for provenance; hard-delete only for legal/abuse reasons, with tombstones.

RESEARCH FINDING. Stack Overflow's own trajectory — a 2023 moderator strike affecting roughly 70% of SO moderators, and community complaints of being "punished for participating" — shows governance tone is an existential variable, not a detail. **INTERPRETATION.** A welcoming, low-blame contribution culture is a competitive weapon precisely because SO is remembered as hostile.

Competitive Analysis

RESEARCH FINDING — the field, by pillar:

Executable procedures: Runme/Stateful (runnable Markdown runbooks, built to fight "bitrot"), Warp Drive notebooks, Doc Detective (tests that docs match reality). All linear or doc-testing; none do branching diagnostic trees with graded evidence. Runme

↗
Doc Detective

↗
AI codebase knowledge: Dosu ("living knowledge base", 50,000+ projects), Unblocked (codebase Q&A), Sourcegraph (code search + "evidence-backed" reasoning traces = citations, not reproduction counts), Glean/Guru (enterprise verified answers + staleness alerts, human-SME "Verified" badges). Dbos + 2

↗
Version-aware docs for AI: Context7 (version-specific docs via MCP), Tessel (spec registry targeting "version blur", founder ex-Snyk). Context7

↗
Q&A/incumbent: Stack Overflow (pivoting to Knowledge-as-a-Service, Stack Overflow for Agents), GitHub Discussions/Issues, DevDocs, Perplexity, Cursor/Copilot.

Dead: Phind (cited developer answers, shut down Jan 2026).

INTERPRETATION — what IS a moat: the **reproduction-evidence graph** (who reproduced what, on which version, with what result) is the one asset no competitor has and that compounds over time;

compatibility data across versions; **diagnostic trees** refined by real branch-outcome data; **contributor reputation earned from reproductions**; and **machine-readable knowledge with an evidence layer** that IDEs/agents could consume (the Stack-Overflow-for-Agents insight, but with evidence attached).

INTERPRETATION — what is NOT a moat: having answers (AI has them), cited answers (Phind died), version-specific docs alone (Context7 does it), executable runbooks alone (Runme does it), or an AI that structures submissions (commodity). Any single pillar is copyable; only the accumulated, hard-to-fake evidence graph resists both LLMs and incumbents.

MVP Research Conclusions

RECOMMENDATION — capability classification:

MVP-CRITICAL: the playbook object (Problem→Test→PASS/FAIL→Fix→Verification); version/environment tagging; author-tested + one-click independent reproduction ("this fixed it" on a stated version); the multi-layer confidence ladder shown as discrete bands with raw counts; hybrid + fingerprint search; hidden-Unicode scanning; DCO + CC BY-SA licensing; append-only evidence bound to immutable revisions.

EARLY GROWTH: AI-assisted structuring/dedup/classification; staleness triggers from dependency releases; multi-environment reproduction; reputation from reproductions; GitHub issue import.

LATER: automated sandbox execution (safe tier); continuous CI re-testing; maintainer-confirmed attestations; IDE/agent API.

EXPERIMENTAL: Docker Compose/browser/edge auto-repro; agent-to-agent evidence exchange.

AVOID: AI-generated verification of any kind; auto-executing untrusted code with network/secrets; global percentage scores on tiny samples; a single "Verified" badge; CLA-based licensing.

INTERPRETATION — the minimum test of the hypothesis. The smallest product that actually tests the thesis is: *a repository where a developer can post a structured fix tagged to a version, and at least one other developer can independently mark "reproduced / failed" on their own version — and where that evidence, not votes, drives what surfaces.* Everything else (AI, sandboxes, trees) is secondary to proving that developers will *contribute reproduction evidence at all.*

Long-Term Opportunities

INTERPRETATION. A machine-readable, evidence-graded knowledge API for AI agents (competing with/complementing Stack Overflow for Agents) — but with reproduction evidence attached, which SO does not have.

Continuously-tested "living playbooks" that re-verify on every dependency release.

Cross-linking with SBOM/security-advisory feeds so a CVE auto-flags every dependent procedure.

A compatibility matrix ("does this fix work on Node 22?") as a standalone data product.

Major Risks

INTERPRETATION. Contribution cold-start — the dominant risk. Without reproductions, the evidence layer is empty and the product is just a worse Stack Overflow.

Reproducibility is intrinsically hard (Jupyter/artifact evidence).

Incumbent fast-follow (SO, Sourcegraph, Dosu, Warp) if the idea proves out.

Abuse/gaming of the evidence system (sockpuppets, fake reproductions).

Security liability from hosting/executing untrusted fixes.

Solo-builder bandwidth against a multi-front technical problem.

Open Questions

UNKNOWN. Will developers contribute reproduction evidence without the dopamine of votes/points? No existing product proves this.

UNKNOWN. What is the minimum viable reproduction — a checkbox, a pasted log, a sandbox run? The trade-off between friction and evidence strength is unresolved.

UNKNOWN. Can staleness triggers be made precise enough to avoid alarm fatigue?

UNKNOWN. Whether an evidence graph can reach useful density in a single-language/single-ecosystem niche before running out of runway.

Research-backed Recommendations

RECOMMENDATION. Pick one narrow, fast-moving, pain-heavy ecosystem to seed (where staleness bites hardest — e.g. a specific edge/runtime or framework) so the evidence graph reaches density before running out of runway. Broad coverage is a trap for a solo builder.

RECOMMENDATION. Make the evidence layer the product, not AI. AI is plumbing; reproduction evidence is the moat. Enforce "AI cannot write verification" at the architecture level.

RECOMMENDATION. Ship the one-click "this fixed it (on version X)" reproduction as the core loop from day one; treat everything else as scaffolding around it.

RECOMMENDATION. Represent confidence as GRADE-style bands with raw counts, never bare percentages; always show failures.

RECOMMENDATION. Bind evidence to immutable revisions; deprecate, never silently edit.

RECOMMENDATION. Use managed microVM/gVisor sandboxes (no network, no secrets, resource caps) if/when auto-execution ships; keep it a supporting signal only.

RECOMMENDATION. Use DCO + CC BY-SA-family licensing; adopt a deliberately welcoming governance tone as an anti-SO differentiator.

RECOMMENDATION. Build hybrid + error-fingerprint search early; it is table stakes for developers pasting stack traces.

RECOMMENDATION — kill criteria. If, after seeding one ecosystem, the ratio of independent reproductions to posted playbooks stays near zero, the core hypothesis has failed and no amount of AI or UX will save it. That ratio is the metric to watch.